

Descritivo Técnico Completo — App Igreja (IBNorte)

Sistema: Aplicativo digital da Igreja Batista Norte

Repositório: `maufreitas63/app-igreja`

Versão do app: 1.0.0

Data deste documento: 2 de julho de 2026

Documentação de entrega: [MANUAL_ENTREGA.md](#) · [INDICE_DOCUMENTACAO.md](#)

1. Resumo executivo

O **app-igreja** é uma plataforma **PWA (Progressive Web App)** e **mobile** (Expo/React Native) que centraliza a operação digital da igreja: login de membros, cadastro e LGPD, dashboard com cards operacionais, check-in em eventos (totem com QR Code, **check-in automático por proximidade/geofence**), gestão familiar, módulo pastoral, financeiro, escalas, mapa de geolocalização por CEP, painel de manutenção administrativa e controle de acesso granular (ACL).

Não há servidor de API próprio: o cliente comunica-se diretamente com o **Supabase** (PostgreSQL, RPCs, RLS, Storage) via HTTPS. O deploy de produção é automático via **Cloudflare Pages** a cada push na branch `main`.

2. Programas, linguagens e ferramentas utilizados

2.1 Runtime e linguagens

Item	Versão / detalhe
Node.js	≥ 20.19.4 (build, scripts, deploy)
TypeScript	~5.9.2 (modo strict)
JavaScript (ESM)	Scripts .mjs de build, importação e testes
SQL (PostgreSQL)	Schema, RPCs, RLS, seeds — execução manual no Supabase
PowerShell	Watchdog Metro LAN (desenvolvimento local)

2.2 Frameworks e bibliotecas principais

Camada	Tecnologia
UI mobile/web	Expo SDK ~54, React 19.1, React Native 0.81.5
Roteamento	Expo Router ~6 (file-based, export estático web)
Backend-as-a-Service	Supabase (@supabase/supabase-js ^2.106)

Formulários	React Hook Form + Zod
Mapas (web)	Leaflet, react-leaflet, markercluster
Mapas (nativo)	react-native-maps, react-native-map-clustering
Estilo	Tailwind CSS, class-variance-authority, Radix UI (web)
Standalone	Vite 6 (formulário público de cadastro familiar)
PDF/docs	md-to-pdf, docx, xlsx (geração de documentação)
Ícones	@expo/vector-icons (FontAwesome, FontAwesome5)

2.3 Infraestrutura e serviços externos

Serviço	Uso
Supabase	Banco PostgreSQL, autenticação customizada via RPC, Storage, Realtime
Cloudflare Pages	Hospedagem HTTPS do PWA (<code>npm run build:web → dist/</code>)
GitHub	Repositório e trigger de deploy
Google Maps Geocoding	Opcional — geocodificação de CEP
ViaCEP + OpenStreetMap	Fallback gratuito de endereço/geolocalização
dailyverses.net	Fonte dos versículos bíblicos por tema (importação via script)
EAS (Expo)	Build/distribuição mobile (project ID configurado em <code>app.json</code>)

2.4 Comandos de build e operação

Comando	Função
<code>npm run build:web</code>	Build de produção (PWA + formulário familiar)
<code>npm start / npm run web</code>	Desenvolvimento local
<code>npm run apply:bible-verses</code>	Aplica scripts SQL de versículos no Supabase
<code>npm run build:docs:pdf</code>	Gera PDFs da documentação existente
<code>npm run lint</code>	ESLint (Expo config)

3. Bancos de dados e modelo de dados

3.1 Supabase (PostgreSQL)

Único banco de produção. Principais domínios de tabelas:

Domínio	Tabelas / objetos (exemplos)
Perfis e sessão	profiles, profile_sessions, RPC verificar_login, issue_profile_session
Família e membros	members, families, sincronização com profiles
Controle de acesso	access_resources, access_roles, access_grants, profile_access_roles
Eventos e check-in	events, checkins, event_favorite_locations, totem, geofence (geofence_ativo), purge triggers
Pastoral	Solicitações, categorias, histórico
Financeiro	Lançamentos, importação, relatórios de despesas
Escalas	Tipos, voluntários, ciclos, vigilância
Geolocalização	cep_geolocation, endereços sincronizados por CEP
Parâmetros	app_parameters (LGPD ativo, totem, etc.)
Versículos	bible_themes, bible_verses_by_theme, RPC get_random_bible_verse
Recepção familiar	Fila de cadastros públicos

3.2 Armazenamento local (dispositivo)

Mecanismo	Dados
AsyncStorage	user_phone, user_profile_id, user_session_token
Sem Supabase Auth session	Cliente configurado com persistSession: false

3.3 Scripts SQL

- **144 arquivos .sql** em `scripts/` (incluindo 15 partes de dados bíblicos).
 - Categorias: ACL (33), ~~perfis/sessão (35)~~, família/membros (35), ~~eventos/check-in/geo (17)~~, financeiro (10), ~~pastoral (7)~~, escalas (8), CEP/geo (8), versículos bíblicos (~18), diagnóstico, seeds de teste (TSTMAX), entre outros.
 - **Geofence (jun/2026):** `events-geofence-ativo.sql` , `event-favorite-locations.sql` , `geo-checkin-automatic.sql` , `geo-checkin-purge-on-event-update.sql` — RPCs atômicas, `normalize_location_key` , triggers de invalidação, RLS restrito em locais favoritos.
 - **Execução:** manual no SQL Editor do Supabase ou via scripts Node (`apply-bible-verses-supabase.mjs`).
-

4. Quantidade de código e linhas

Métricas levantadas em **17/06/2026** (excluindo `node_modules` , `dist` , `.expo` , `package-lock.json`):

4.1 Visão geral

Métrica	Valor
Arquivos no repositório	~715
Arquivos TypeScript/TSX	287
Linhas TS/TSX (código aplicativo)	61.517
Arquivos SQL	144
Linhas SQL (lógica/schema/RPC)	25.031
Linhas SQL (dados bíblicos)	11.067
Linhas SQL (total)	~ 36.098
Scripts Node (.mjs)	35 arquivos / 4.264 linhas
Documentação Markdown	32 arquivos / ~ 11.986 linhas

4.2 Por camada / pasta

Pasta	Arquivos	Linhas (aprox.)
app/ (rotas Expo Router)	22	20.574
components/	67	17.694
lib/ (regras de negócio)	147	17.635
hooks/	43	5.335
scripts/ (total)	190	—
standalone/cadastro-familia/	5	incluído em TS

4.3 Dependências npm

Tipo	Quantidade
Produção	60 pacotes
Desenvolvimento	16 pacotes

Total estimado de linhas de código-fonte relevante (TS + SQL lógica + scripts): ~91.000 linhas
(com dados bíblicos em SQL: ~102.000 linhas)

5. Arquitetura em camadas

Camada 1 – Dispositivo (PIN, AsyncStorage, câmera, totem)
Camada 2 – Cliente (telas, guards ACL, filtros de cards)
Camada 3 – Transporte (HTTPS, headers de sessão, anon key)
Camada 4 – Supabase (RLS, RPC SECURITY DEFINER, grants)

5.1 Camada de apresentação (UI)

- **Rotas:** `app/` — Expo Router (file-based).
- **Componentes:** `components/` — cards do dashboard, painéis de manutenção, formulários, mapa, UI compartilhada.
- **Contexto:** `context/EntityPrefixContext.tsx` — prefixo de entidade para IDs.
- **Constantes visuais:** `constants/theme.ts` .

Telas principais:

Rota	Função
<code>/</code>	Login (telefone + PIN de 4 dígitos)
<code>/register</code>	Cadastro inicial de perfil
<code>/(tabs)/index</code>	Índice de atalhos do membro
<code>/(tabs)/dashboard</code>	Dashboard com carrossel de cards
<code>/maintenance-dashboard</code>	Painel administrativo
<code>/manage-profile</code>	Dados cadastrais
<code>/manage-members</code>	Gestão de membros da família
<code>/pastoral, /pastoral-history</code>	Módulo pastoral
<code>/financial, /expense-report</code>	Módulo financeiro
<code>/mapa-geolocalizacao</code>	Mapa por CEP
<code>/totem-checkin</code>	Check-in em totem (QR/câmera)

/lgpd	Aceite LGPD
/cadastro-familia	Formulário público (standalone Vite)
/sessao-encerrada	Tela pós-logout no PWA instalado

5.2 Camada de hooks

- `hooks/` — 43 arquivos: guards de ACL (`useScreenAccessGuard`), dados de manutenção, check-in, financeiro, PWA install, eventos selecionados.

5.3 Camada de domínio / serviços (`lib/`)

- **147 módulos** com regras de negócio puras e integração Supabase.
- Exemplos: `accessControl.ts` , `userSession.ts` , `verificarLogin.ts` , `family*.ts` , `pastoralRequest.ts` , `financial*.ts` , `geoMapGeocoding.ts` , `bibleVerseByTheme.ts` .
- **Sem servidor intermediário** — chamadas diretas `supabase.from()` e `supabase.rpc()` .

5.4 Camada de dados (servidor)

- PostgreSQL no Supabase com **RLS** habilitado nas tabelas sensíveis.
- **RPCs SECURITY DEFINER** para operações que exigem validação centralizada.
- Políticas de leitura/escrita baseadas em `profile_has_access` e papéis (`access_roles`).

5.5 Camada de tooling (`scripts/`)

- Geração de build info, ícones PWA, importação de versículos, aplicação SQL, testes automatizados de fluxos, geração de documentação PDF/XLSX, seeds de teste.

5.6 Aplicação standalone

- `standalone/cadastro-familia/` — formulário Vite/React reutilizando `FamilyRegistrationForm` .
- Publicado em `/cadastro-familia/` no mesmo domínio Cloudflare.
- Submissão via RPC pública `submit-family-registration-public` .

6. Segurança da aplicação

6.1 Modelo de defesa em profundidade (4 camadas)

Documentado em `CAMADAS_SEGURANCA.md` :

1. **Dispositivo** — PIN de 4 dígitos validado no servidor; sessão em AsyncStorage; totem isolado do fluxo de membro; permissões de câmera explícitas.
2. **Cliente** — Guards de tela (`sessionHasAccess`); cards do dashboard filtrados por ACL; colunas sensíveis com ACL de coluna; modo `EXPO_PUBLIC_ACL_STRICT=true` (fail-closed se RPC de ACL ausente); `TotemDeviceRouteGuard` ; **cache em memória** de ACL e perfil (`lib/asyncResultCache.ts`) para navegação instantânea entre cards.
3. **Transporte** — HTTPS; header `x-session-token` ou `x-profile-id` em toda requisição; apenas chave **anon** no app (sem `service_role`).
4. **Servidor** — RLS; RPCs com validação de permissão; grants por papel; escritas sensíveis somente via RPC.

6.2 Controle de acesso (ACL)

Nível	Exemplo	Onde se aplica
Tela	<code>screen:/financeiro</code>	Guard de rota
Card	<code>screen:dashboard.card.financeiro</code>	Filtro do carrossel
Tabela	<code>table:profiles</code>	RLS
Coluna	<code>column:profiles.access_pin</code>	Dados cadastrais

6.3 Rotas públicas (sem guard de ACL)

`/` , `/register` , `/totem-checkin` , `/cadastro-familia` , `/sessao-encerrada`
 — documentado em `docs/SECURITY-PUBLIC-ROUTES.md` .

6.4 Sessão e autenticação

- Login: telefone + PIN → RPC `verificar_login` .
- Token de sessão: `issue_profile_session` → header `x-session-token` .
- Logout: limpa AsyncStorage e redireciona com `?signedOut=1` .

6.5 Headers HTTP (Cloudflare)

- HTML: `Cache-Control: must-revalidate` (atualizações de deploy).
- Assets com hash: cache longo.
- `X-Content-Type-Options: nosniff` .

6.6 Dados sensíveis

- PIN, CPF, alertas médicos: protegidos por ACL de coluna + RPC de escrita.
- LGPD: fluxo de aceite obrigatório quando parâmetro ativo.

6.7 Riscos e boas práticas observadas

Aspecto	Situação
---------	----------

Chave anon no código	Valores padrão em lib/supabaseConfig.ts — esperado para cliente; segurança depende de RLS
Scripts SQL	Execução manual — requer governança de quem aplica no Supabase
Service role	Não embutida no app
PWA instalado	Sessão persiste localmente — protegida pelo PIN

7. Fluxo de deploy

Alterações locais → git commit → git push origin main → GitHub → Cloudflare Pages
→ npm install → npm run build:web → publicação em dist/ (HTTPS)

Configuração Cloudflare	Valor
Build command	npm run build:web
Output directory	dist
Node	20
Branch	main

O build gera `public/build-info.json` com hash do commit para rastreabilidade.

8. Funcionalidades por módulo

Módulo	Descrição
Acesso e sessão	Login PIN, restauração de sessão, logout, reparo de referência de perfil
LGPD	Aceite de termos, bloqueio de fluxo se pendente
Dashboard membro	Carrossel: eventos, QR check-in, kids/teens, ofertas, pastoral, membros, aniversariantes, financeiro, escalas, estacionamento
Índice	Atalhos filtrados por ACL; versículo bíblico aleatório por tema
Manutenção	Gantt de eventos, quorum, escalas, ACL, financeiro, pastoral, recepção familiar, monitor de salas
Família	CRUD membros, transferência entre famílias, reconhecimento familiar
Check-in / Totem	QR Code, câmera, confirmação de audiência familiar
Pastoral	Abertura e histórico de solicitações

Financeiro	Visão mensal, boletins, relatórios de despesas
Mapa	Geolocalização por CEP com clusters (Leaflet web)
Versículos	161 temas, 5.247 versículos (dailyverses.net)
Cadastro público	Formulário familiar sem login (fila recepção)

9. Documentação existente no repositório

Documento	Conteúdo
ARQUITETURA_BLUEPRINT_PWA.md	Blueprint técnico completo
CAMADAS_SEGURANCA.md	Especificação de segurança
CONTROLE_ACESSO.md	Modelo ACL
DEPLOY_CLOUDFLARE.md	Guia de deploy
FUNCIONALIDADES.md	Lista de funcionalidades
PACOTE_* / manuais	Pacotes operacionais e treinamento

10. Informações complementares

- **Nome PWA:** Igreja Batista Norte (IBNorte)
- **Orientação:** Portrait
- **Plataformas alvo:** Web (PWA), Android, iOS (via Expo)
- **Repositório Git:** <https://github.com/maufreitas63/app-igreja>
- **Testes automatizados:** Scripts Node em `scripts/test-*.mjs` (fluxos específicos, não suite Jest completa)
- **Geração de documentação:** `npm run build:docs:pdf` converte ~27 markdowns em PDF na pasta `pdfs/`

Documento gerado automaticamente com base na análise do repositório app-igreja em 17/06/2026.